Lab Sheet 7

Develop a complete Django app mapping model array collections out to unordered item matrices (Fruits)alongside ordered indices (Selected Event Students).
Task 7.1: Implement robust fallback condition logic filters inside the Django template layer ({% if %}) to show an alert if arrays arrive empty.
Task 7.2: Incorporate dynamic table sorting tools inside the template views to order records based on specific field properties (e.g., student name).
Task 7.3: Build a simple user search input form that screens long array matrices and subsets lists dynamically using back-end request processing rules.

views.py

```
from django.shortcuts import render
def fruits(request):
    fruits = ["Apple", "Mango", "Banana", "Orange", "Grapes"]

    return render(request, "fruits.html", {
        "fruits": fruits
    })
def students(request):
    students = [
        {"name": "Rakul", "event": "Coding"},
        {"name": "Divyashi", "event": "Quiz"},
        {"name": "Sakshi", "event": "Hackathon"},
        {"name": "Priyashi", "event": "Debate"},
        {"name": "Sneha", "event": "Coding"},
        {"name": "Arprita", "event": "Quiz"},
    ]

    query = request.GET.get("q", "")

    if query:
        students = [
            student for student in students
            if query.lower() in student["name"].lower()
        ]

    sort = request.GET.get("sort", "name")

    if sort == "name":
        students = sorted(students, key=lambda x: x["name"])

    elif sort == "event":
        students = sorted(students, key=lambda x: x["event"])

    return render(request, "students.html", {
        "students": students,
        "query": query,
    })
```

urls.py

```
from django.contrib import admin
from django.urls import path
from app import views

urlpatterns = [
    path("admin/", admin.site.urls),
    path("fruits/", views.fruits),
    path("students/", views.students),
]
```

fruits.html

```
<!DOCTYPE html>
```

```
<html>
<head>
   <title>Fruits</title>
</head>
<body>
<h1>Fruit List</h1>
{% if fruits %}
   <ul>
      {% for fruit in fruits %}
         <li>{{ fruit }}</li>
      {% endfor %}
   </ul>
{% else %}
   <p>Alert: No fruits are available.</p>
{% endif %}
</body>
</html>
```

students.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Event Students</title>
</head>
<body>
<h1>Selected Event Students</h1>
<form method="GET">
   <input
      type="text"
      name="q"
      placeholder="Search student"
      value="{{ query }}"
```

Fruit List

- Apple
- Mango
- Banana
- Orange
- Grapes

If the fruits array is empty:
Alert: No fruits are available.

Selected Event Students

```
Student Name        Event
--------------------------------
Arprita             Quiz
Divyashi            Quiz
Priyashi            Debate
Rakul               Coding
Sakshi               Hackathon
Sneha                Coding
```

The student records are displayed in alphabetical order
when the default sort value is "name".

Selected Event Students

Search student: [priyashi] [Search]

```
Student Name        Event
--------------------------------
    priyashi       Debate
```

The backend filters the student list using the entered
query and displays only matching names.

Fruits page:
http://127.0.0.1:8000/fruits/

Students page:
http://127.0.0.1:8000/students/

Example search:
http://127.0.0.1:8000/students/?q=Priyashi

The Django pages render the requested arrays, apply the
fallback condition, sort records, and filter students.

The Django application successfully demonstrates:
1. Conditional fallback handling for empty arrays.
2. Dynamic sorting of student records by field.
3. Backend search and filtering using a GET request.